

Universität Stuttgart
Master of Science

Networking basics

Marco Aiello

Networking

Networking is *“the action or process of interacting with others to exchange information and develop professional or social contacts.”* for computers the information is digital and the goal is coordination of computations.

Telecommunication network based on physically wired, optical and wireless radio frequency links.

Network topology is physical and logical arrangement of the nodes and the links of the network.

Networking

Basic definitions

- *Latency*: difference between starting a transmission and receiving the first bit on the other end
- *Data transfer rate*: amount of data transferred in the unit of time
- *Bandwidth*: maximum amount of data that can be sent in the unit of time

$$\text{Message transmission time} = \text{latency} + \frac{\text{message size}}{\text{data transfer rate}}$$

Networking

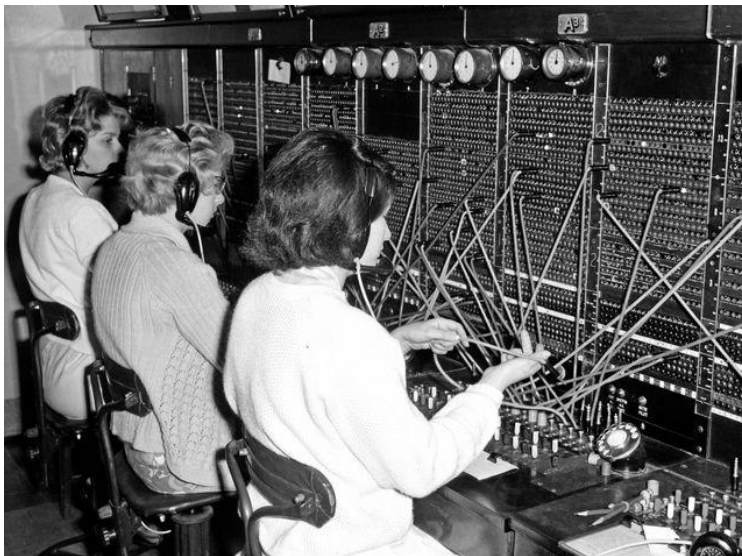
Types of networks

		<i>Example</i>	<i>Range</i>	<i>Bandwidth (Mbps)</i>	<i>Latency (ms)</i>
<i>Wired:</i>					
LAN		Ethernet	1–2 kms	10–10,000	1–10
WAN		IP routing	worldwide	0.010–600	100–500
MAN		ATM	2–50 kms	1–600	10
Internetwork		Internet	worldwide	0.5–600	100–500
<i>Wireless:</i>					
WPAN		Bluetooth (IEEE 802.15.1)	10–30m	0.5–2	5–20
WLAN		WiFi (IEEE 802.11)	0.15–1.5 km	11–108	5–20
WMAN		WiMAX (IEEE 802.16)	5–50 km	1.5–20	5–20
WWAN		3G phone	cell: 1–5	348–14.4	100–500

Networking

Principles

- *Packet transmission*: message divided into data packets of equal size
- *Package delivery*:
 - Broadcast: send to all, receiver decides whether to consume the message
 - Circuit switching: physical change in the network topology to connect process that need to exchange messages
 - Packet switching: store-and-forward method, routing policies define the “forward”
frame-relay is specific type of packet switching



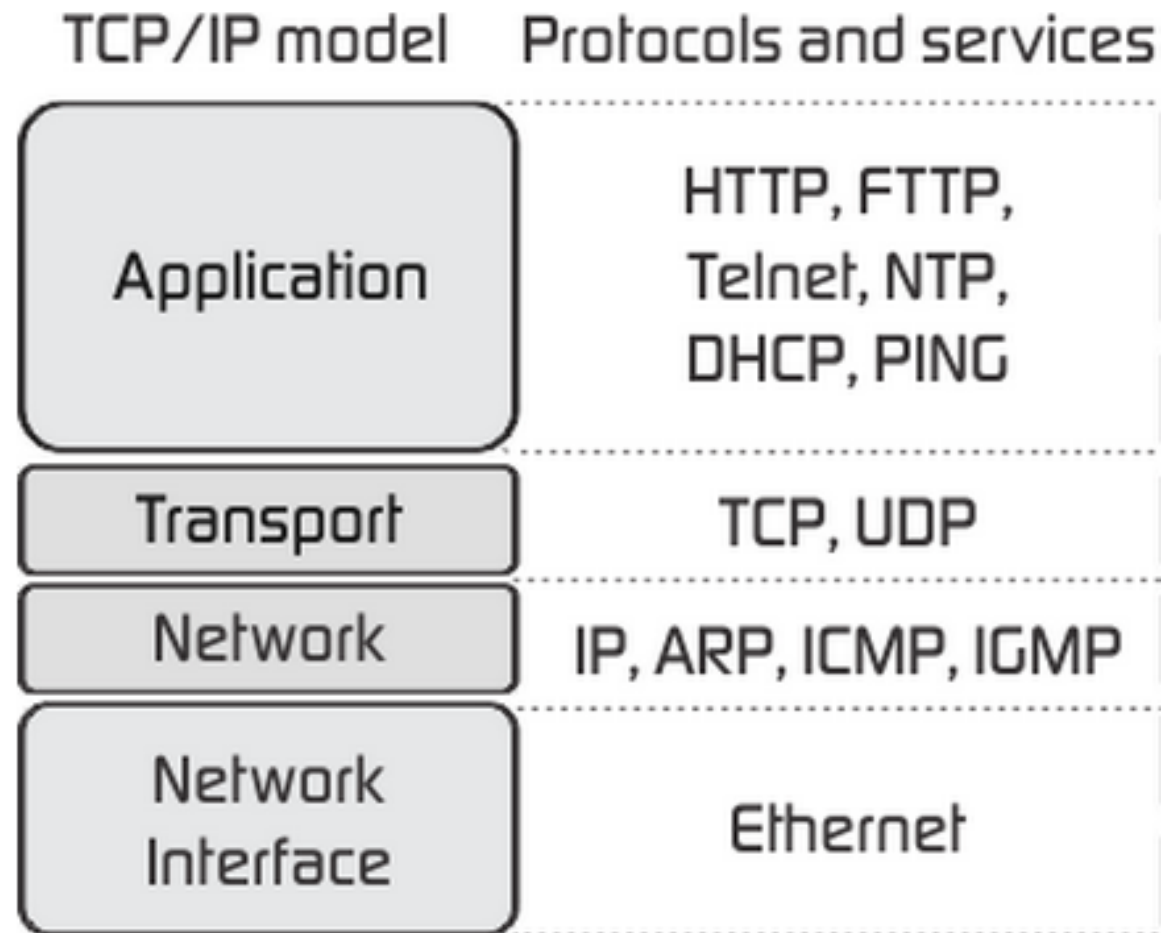
Networking

ISO/OSI Protocol Stack

Layer			Protocol data unit (PDU)	Function
Host layers	7	Application	Data	High-level protocols such as for resource sharing or remote file access, e.g. HTTP.
	6	Presentation		Translation of data between a networking service and an application; including character encoding, data compression and encryption/
	5	Session		Managing communication sessions, i.e., continuous exchange of information in the form of multiple back-and-forth transmissions
	4	Transport	Segment, Datagram	Reliable transmission of data segments between points on a network, including segmentation, acknowledgement and
Media layers	3	Network	Packet	Structuring and managing a multi-node network, including addressing, routing and traffic control
	2	Data link	Frame	Transmission of data frames between two nodes connected by a physical layer
	1	Physical	Bit, Symbol	Transmission and reception of raw bit streams over a physical medium

Networking

TCP/IP and ISO/OSI model



Networking

Routing

Routing is a distributed algorithm that defines the path a message needs to take to go from origin to destination.

A logical overlay on top of a (physical) network

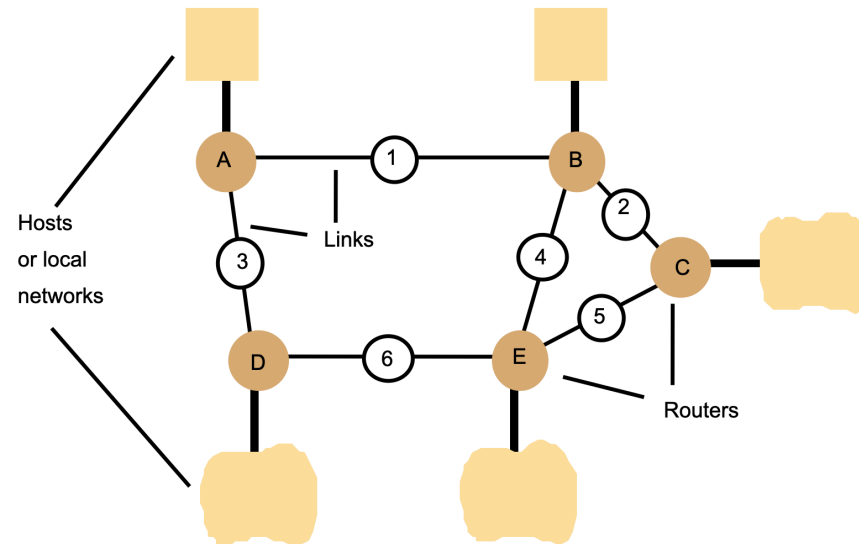
Trivial approach: flooding

Better approach: explore and find minimal paths

Update upon new information (new edge, new node, failure)

Networking

Routing example



Routings from A			Routings from B			Routings from C		
To	Link	Cost	To	Link	Cost	To	Link	Cost
A	local	0	A	1	1	A	2	2
B	1	1	B	local	0	B	2	1
C	1	2	C	2	1	C	local	0
D	3	1	D	1	2	D	5	2
E	1	2	E	4	1	E	5	1

Routings from D			Routings from E		
To	Link	Cost	To	Link	Cost
A	3	1	A	4	2
B	3	2	B	4	1
C	6	2	C	5	1
D	local	0	D	6	1
E	6	1	E	local	0

Internetworking

The TPC/IP stack

Basis of the Internet

Before ISO/OSI model

Store and forward principle

Internet Protocol (IP): defines a universal addressing scheme

ISO/OSI: level 3

Addresses are divided in four classes of addresses, A,B,C,D

The protocol is used for the transfer of datagrams from one host to another possibly using intermediate nodes.

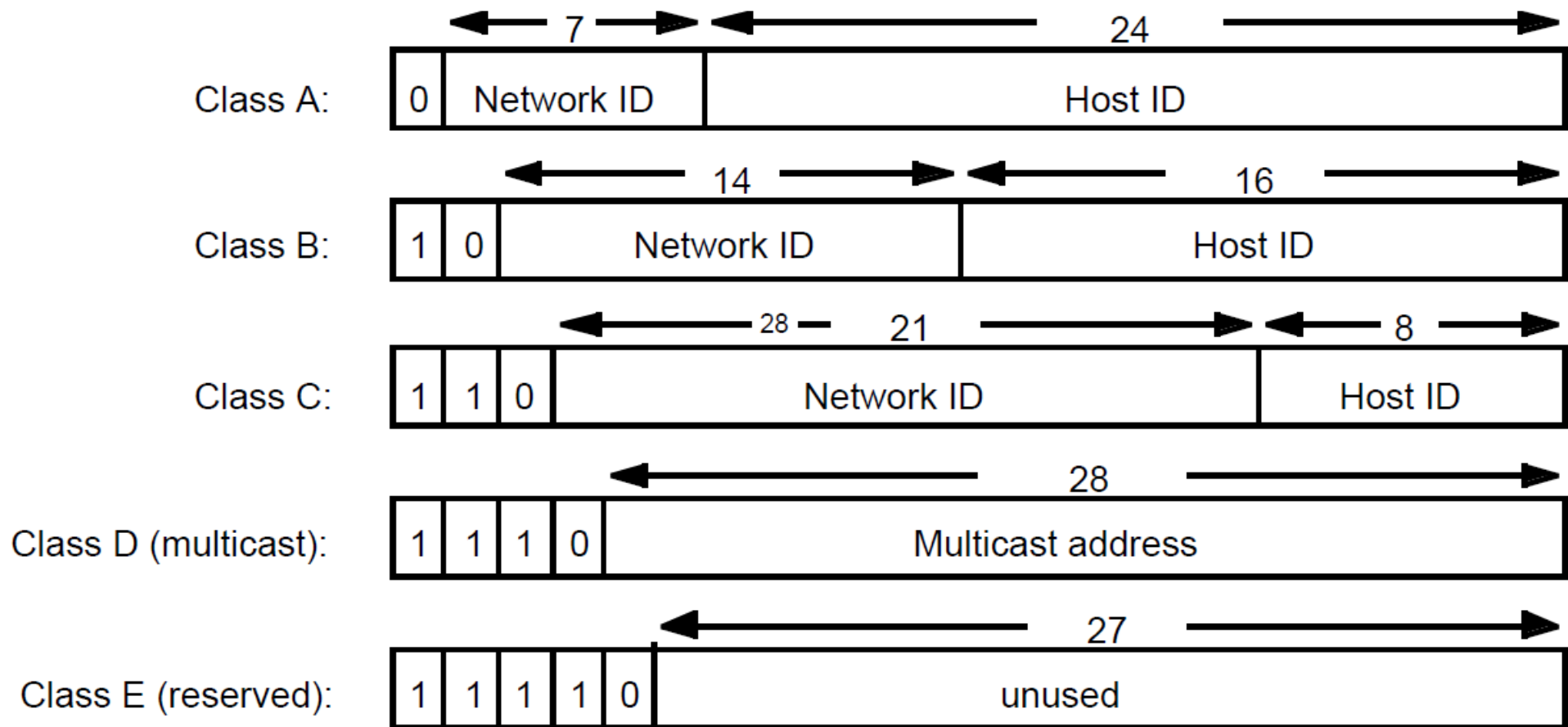
Best effort semantics

Header checksum (just for the address, not for the data)

Uses the Address Resolution Protocol (ARP)

Internetworking

The TCP/IP stack

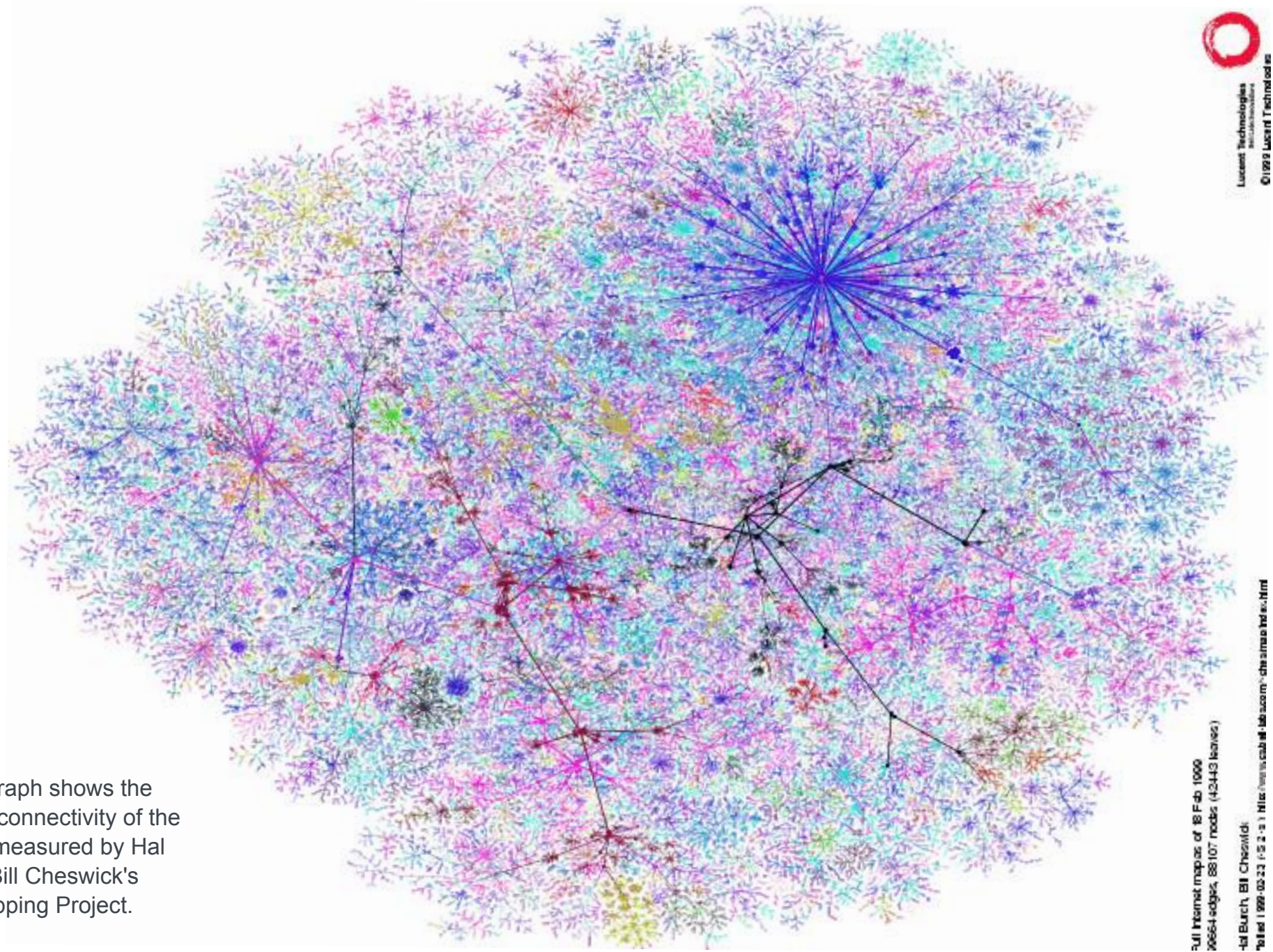


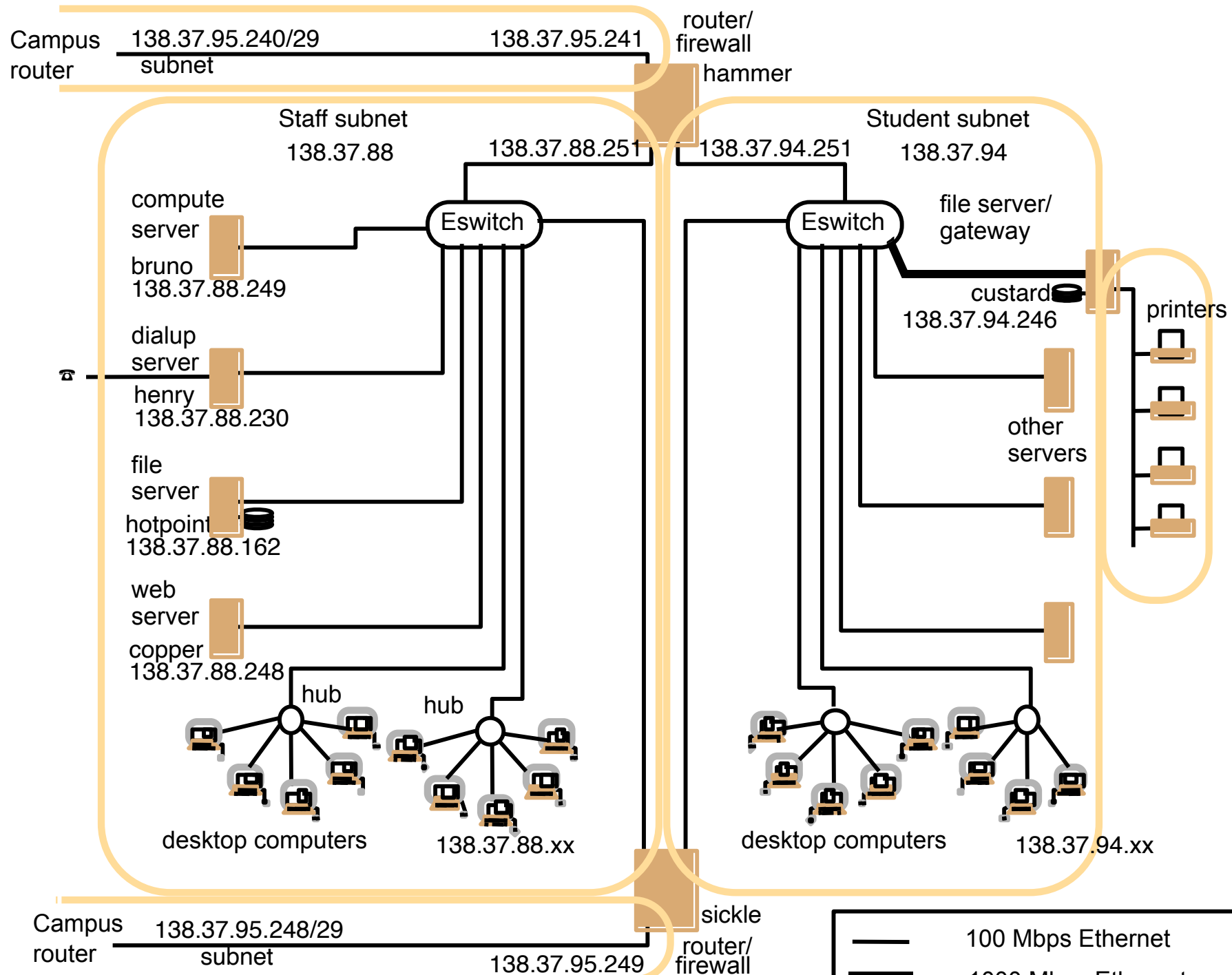
Internetworking

The TPC/IP stack

	octet 1	octet 2	octet 3		Range of addresses
	Network ID		Host ID		
Class A:	1 to 127	0 to 255	0 to 255	0 to 255	1.0.0.0 to 127.255.255.255
	Network ID		Host ID		
Class B:	128 to 191	0 to 255	0 to 255	0 to 255	128.0.0.0 to 191.255.255.255
	Network ID		Host ID		
Class C:	192 to 223	0 to 255	0 to 255	1 to 254	192.0.0.0 to 223.255.255.255
	Multicast address				
Class D (multicast):	224 to 239	0 to 255	0 to 255	1 to 254	224.0.0.0 to 239.255.255.255
Class E (reserved):	240 to 255	0 to 255	0 to 255	1 to 254	240.0.0.0 to 255.255.255.255

This large graph shows the router level connectivity of the Internet as measured by Hal Burch and Bill Cheswick's Internet Mapping Project.





User Datagram Protocol

UDP/IP stack

- Process to process communication at the transport level
- Port numbers to identify processes
- A datagram encapsulated inside an IP packet
- No delivery guarantees (omissions, un-ordered, un-integer)
- Basically a transmission level IP protocol
- Used for class D addresses

Transmission Control Protocol

TPC/IP stack

- A reliable, flow-oriented transmission protocol
- ISO/OSI: level 4
- Divide message in a sequence of data segments = IP Packets
- Attach a sequence number to each segment
- Flow control: by segment acknowledgements
- Retransmission of missing segments
- Ordering of segments at receiver's end
- Checksum of each segment
- Buffering. When buffer is full stop sending acknowledgements to force retransmission

TCP datagram reliability vs. Application Message Reliability

or *“I don’t need reliable messaging, I use TCP”*

- TCP ensures datagram reliable delivery
- Application-level messages use some form of transport, often multi-hop and multi-path
- Saying *“I don’t need reliable messaging, I use TCP”* is like saying *“I don’t need a ABS, I have a break pedal”*

Application messaging faults despite TCP

Issues

1. **Application Crashes or Restarts Mid-Message:** the application crashes after receiving but before putting it in hold-back queue
2. **Duplicate Messages:** TCP guarantees delivery, not idempotency, if the sender retries due to a timeout, duplicate messages are possible
3. **Connection Drops:** TCP handles reconnection, but messages in transit during disconnect may be lost, and there is no built-in message queueing or re-delivery after reconnect
4. **Backpressure & Overload:** TCP has flow control but doesn't help with application-level backpressure. The application might accept messages via TCP but fail to process them due to overfull buffers

Application messaging faults despite TCP

Examples

1. A payment service gets a "charge card" message via TCP, crashes after debiting the account but before logging it as processed — now you risk *double charging* if it's retried.
2. Client sends “place order” message. Server gets it and processes, but the client doesn't get a response due to a network hiccup — it retries. Now the server receives two identical orders.
3. Client sends a series of messages; connection drops halfway — some were received, some not, but one doesn't know which.
4. A TCP-based log shipper keeps sending messages, but the receiver can't write them fast enough — memory fills up, application buffer overflow.

Route changes, even with TCP

1. *Out-of-order delivery* (across multiple TCP links)
 - TCP maintains order per connection, not globally
 2. *Multiple paths*
 - load balancers distributing messages
 - path randomization
 - link failure
- **When to Worry:**
 - load balancing across multiple backend nodes
 - building real-time systems that need message ordering
 - dealing with multi-hop networks, proxies, or complex routing

How can an application message surpass another one using TCP sockets?

1. **Segmentation and Reassembly at the Transport Layer:** TCP may segment large messages into smaller packets and send them separately. If two application messages are sent, they may be split into different TCP segments, and network conditions may cause one to be delayed while another one arrives first.
2. **Nagle's Algorithm:** TCP uses Nagle's Algorithm to reduce small packet transmission by buffering data before sending. If one message is small and another is large, the smaller one might be sent first due to less buffering delay.
3. **Multiple TCP Connections:** A message sent on one connection may arrive before another one sent on a different connection.
4. **Application-Level Buffering and Processing Order:** Even if TCP delivers data in order, the application might process messages in a different order due to:
 1. Thread scheduling in multi-threaded applications.
 2. Non-blocking sockets, where the application reads messages in an unpredictable sequence.
 3. Processing delays, where one message requires more processing time than another.
5. **Prioritization at the Application Layer:** If the sender or receiver implements priority queuing, it may prioritize certain messages over others. Example: A control message (e.g., "PING") may be processed before a large data transfer message.